

☰ Hide menu

About this Course

Key Components of Llamafile

- ✓

Reading: Key Terms
5 min
- ✓

Reading: What is Llamafile?
5 min
- ✓

Video: Llamafile overview by Mozilla
5 min
- ✓

Video: Using the Llamafile API
2 min
- ✓

Video: Creating a Llamafile
5 min
- ✓

Reading: Cosmopolitan
5 min
- ✓

Video: Building portable binaries with Cosmopolitan
4 min
- 📅

Lab: Cosmopolitan
10 min
- 📖

Reading: Lesson Reflection
5 min
- 📋

Graded Assignment: Quiz- Key Components of Llamafile
10 min

Developing Portable CLI with Cosmopolitan

Running Llamafile

Conclusion and Next Steps

Cosmopolitan

🔗 Launch lab

Instructions

Lab Goal

Apply concepts from lesson by creating and running a simple Marco Polo command line program.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main(int argc, char *argv[]) {
5      if (argc > 1 && strcmp(argv[1], "Marco") == 0) {
6          printf("Polo\n");
7      } else {
8          printf("Usage: marco-polo Marco\n");
9      }
10
11     return 0;
12 }
```

Run

Reset

Usage: marco-polo Marco

Lab Description & Steps

1. Use cosmocc compiler to create marco-polo.c program
2. Build program with missing include statement to see compiler warning
3. Fix code and rebuild marco-polo properly
4. Run program correctly and incorrectly to print output
5. Experiment with modifying code and observing behavior

Reflection Questions

1. What did you learn about the C build process and runtime?
2. What role did compiler warnings play in debugging?
3. How did command line arguments modify program flow?
4. What libraries were required for basic input/output?
5. How might you expand the program's capability?

Challenge Exercises

1. Add second command line argument to print different output
2. Redirect program output to a file
3. Handle invalid arguments without crashing
4. Expand to echo any command line argument
5. Modify code to take input from user instead of arguments

Share My Work

Generate or update a read-only copy of your lab. [Learn more](#)

Generate/Update shared link